

# Podstawy Symulacji Komputerowej

## Lista 1 zadanie 2

### Treść zadania

Napisać program wykonujący algorytm kwadratowy von Neumanna.

### Opis metody

Algorytm środka kwadratu (zaproponowany przez Johna von Neumanna) to jedna z najstarszych metod generowania liczb pseudolosowych. Metoda opiera się na następujących krokach:

1. Wybór początkowej wartości ziarna (ang. *seed*) o długości  $n$  cyfr.
2. Podniesienie tej wartości do kwadratu, co daje liczbę o długości do  $2n$  cyfr.
3. Uzupełnienie wyniku zerami wiodącymi (jeśli to konieczne), aby zachować stałą długość  $2n$ .
4. Wyciągnięcie  $n$  środkowych cyfr, które stają się kolejną wygenerowaną liczbą i nowym ziarnem dla następnej iteracji.

Częstym problemem tego algorytmu jest szybkie zapętlenie się lub zbieganie do zera. W poniższej implementacji wprowadzono mechanizm zapobiegający temu zjawisku poprzez resetowanie wartości w przypadku osiągnięcia zera.

### Implementacja

Symulację wykonano w języku Python. Poniżej przedstawiono implementację generatora oraz funkcji wizualizujących niezależność wygenerowanych próbek w przestrzeni 2D i 3D:

```
1 import matplotlib.pyplot as plt
2
3 def von_neumann_generator(seed, n_samples):
4     current = seed
5     results = []
6
7     for i in range(n_samples):
8         # Podniesienie do kwadratu i wyrownanie do 8 znakow
9         square_str = str(current**2).zfill(8)
10
11         # Pobranie 4 srodkowych cyfr (indeksy 2,3,4,5)
12         middle_str = square_str[2:6]
13         current = int(middle_str)
14
15         # Normalizacja do przedzialu [0, 1)
16         results.append(current / 10000.0)
17
18         # Zabezpieczenie przed utknieciem w zerze
19         if current == 0:
20             current = 1234 + i
21
22     return results
```

```

23
24 def plot_2d_independence(data):
25     x = data[:-1]
26     y = data[1:]
27
28     plt.figure(figsize=(8, 8))
29     plt.scatter(x, y, alpha=0.6, s=15, color='blue')
30     plt.title("Test graficzny 2D: Von Neumann ($r_n$ vs $r_{n+1}$)")
31     plt.xlabel("$r_n$")
32     plt.ylabel("$r_{n+1}$")
33     plt.grid(True, linestyle='--', alpha=0.5)
34     plt.show()
35
36 def plot_3d_independence(data):
37     # Tworzymy trojki (x, y, z)
38     x = data[:-2]
39     y = data[1:-1]
40     z = data[2:]
41
42     fig = plt.figure(figsize=(10, 8))
43     ax = fig.add_subplot(111, projection='3d')
44
45     ax.scatter(x, y, z, alpha=0.6, s=10, color='purple')
46     ax.set_title("Test graficzny 3D: Von Neumann ($r_n, r_{n+1}, r_{n+2}$)")
47     ax.set_xlabel("$r_n$")
48     ax.set_ylabel("$r_{n+1}$")
49     ax.set_zlabel("$r_{n+2}$")
50     plt.show()
51
52 # Wykonanie algorytmu
53 samples = von_neumann_generator(seed=1234, n_samples=500)

```

Listing 1: Implementacja algorytmu kwadratowego von Neumanna

## Wyniki symulacji i testy graficzne

Wygenerowano próbkę  $N = 500$  liczb pseudolosowych dla początkowego ziarna 1234. Pierwsze wartości wygenerowanego ciągu to m.in.: 0.5227, 0.3215, 0.0336, 0.1128.

Aby sprawdzić jakość wygenerowanego ciągu i niezależność kolejnych próbek, przeprowadzono testy graficzne w wymiarze 2D (punkty o współrzędnych  $r_n, r_{n+1}$ ) oraz 3D (punkty o współrzędnych  $r_n, r_{n+1}, r_{n+2}$ ).

Rysunek 1: Rozkład par  $(r_n, r_{n+1})$  w teście 2D.

Rysunek 2: Rozkład trójek  $(r_n, r_{n+1}, r_{n+2})$  w teście 3D.

## Interpretacja wyników

Na podstawie przeprowadzonych testów graficznych (wykresy 2D i 3D) można wstępnie ocenić jakość generatora. Zbyt wyraźne wzory, prążki lub skupiska punktów na wykresach świadczyłyby o silnej zależności między kolejnymi próbkami, co jest charakterystyczną wadą prostych generatorów (takich jak klasyczny algorytm von Neumanna). Zastosowane w kodzie zabezpieczenie (zmiana stanu przy wygenerowaniu 0) zapobiega całkowitej degradacji ciągu, jednak dla profesjonalnych zastosowań symulacyjnych metoda ta ustępuje nowoczesnym generatorom.